

EE 381: EE LABORATORIES (DIGITAL CIRCUITS AND MICROPROCESSORS) (2025-2026/II)

EXPERIMENT 7: FPGA EXPERIMENTS USING EDGE SPARTAN 7 FPGA DEVELOPMENT BOARD

Introduction

The aim of this experiment is to perform basic digital logic operations using **Field Programmable Gate Arrays (FPGAs)**. Digital circuits will be designed, implemented, and tested on the **Xilinx Spartan-7 FPGA** using **Verilog HDL** and **Vivado Design Suite**.

LOGIC INTERFACE ON EDGE SPARTAN 7 FPGA DEVELOPMENT BOARD

The EDGE Spartan 7 FPGA development board is a configurable digital logic system. It is powered by the **Xilinx Spartan-7 FPGA (XC7S15FTGB196-1)**, which enables users to implement a wide range of digital circuits using **Verilog HDL** and the **Vivado Design Suite**. In this experiment, we will explore the digital I/O functionalities of the FPGA development board to perform fundamental logic operations and interface designs. Some key features:

1. The FPGA development board has **16 slide switches** and **16 discrete LEDs**, which serve as digital input and output lines and each switch can be set to either '0' or '1', corresponding to a low or high logic level. The status of each digital output is indicated by LEDs, allowing real-time visualization of logic states. Keep the jumpers J4 and J9 to the ground side (E) to light the LEDs from the FPGA pins.
2. The switches and LEDs are connected to specific FPGA I/O pins, which can be configured through Verilog code in Vivado. The FPGA development board is programmed using **Vivado Design Suite** with **Verilog HDL**, and the bitstream is dumped via the **USB interface** for real-time testing.
3. The FPGA development board also includes a **4-digit seven-segment display**.
4. The **Spartan-7 FPGA (XC7S15)** on the FPGA development board features **12.8K logic cells**, **20 DSP blocks**, **360 Kb of Block RAM**, and **two clock management tiles**, enabling complex logic designs, arithmetic operations, high-speed memory processing, and flexible clock signal generation.

Using the above features one can implement a variety of Verilog codes.

PART 1: Realization of Arithmetic Circuits (Full Adder and Full Subtractor)

Design and implement a **Full Adder** and a **Full Subtractor** circuits using Verilog HDL. Simulate the designs in Vivado to verify their truth tables and functionality. The bitstream will be generated and deployed on the FPGA development board to test the circuits using slide switches and LEDs.

1.1 Lab Preparation

1. Design the full adder and full subtractor making use of appropriate input variables and outputs. Verify your design using suitable truth tables and write them down.
2. Draw the complete circuit diagrams for full adder and full subtractor.
3. Write down the Verilog code for each circuit.

1.2 Experiment

1. Write Verilog codes for Full Adder and Full Subtractor. Use **Boolean expressions** and **gate-level modeling** to implement the circuits.
2. Simulate the designs in Vivado to verify the truth tables and functionality of each circuit. Confirm that the outputs correspond to the expected results.
3. Synthesize the Verilog code and generate the bitstream file. Check for synthesis errors and ensure the design is successfully implemented.

4. Dump the bitstream onto the FPGA development board. Use slide switches to provide binary inputs for the circuits. Observe the outputs on the LEDs, verifying the sum and carry (for adders) and difference and borrow (for subtractors).

PART 2: 7-Segment Display Control Using Verilog

Design and implement a Verilog code to control the 7-segment display on the FPGA development board. The digits (0-F) will be displayed based on input from slide switches. The segments of the display will be mapped to the corresponding hexadecimal values, enabling real-time visualization of digital data.

2.1 Lab Preparation and Experiment

1. Develop and write Verilog code to display hexadecimal digits (0-F) on the 7-segment display also write the truth table.
2. **Common anode lines** for each digit on the 7-segment display are **assigned to FPGA pins (H4, H3, H2, H1)** using **Vivado**.
3. **Individual segments (A to G, and DP)** for each digit on the 7-segment display are **assigned to FPGA pins (L3, P4, P2, M3, M1, J4, K4, J2)** using **Vivado**. A zero is to be written to a cathode pin to light the segment led.

OPTIONAL

PART 3: Basic Logic Operations Using Switches (AND, OR, NOT, NOR, NAND, XOR, XNOR)

Design and implement all basic logic gates using a **single Verilog code**. Configure the slide switches on the FPGA development board to select the desired gate. Input values are provided through switches, and the outputs are displayed on LEDs. The experiment demonstrates dynamic control of logic operations, and users can verify truth tables and observe the behavior of each gate.

3.1 Lab Preparation

1. Write a unified Verilog code that implements all mentioned gates.
2. Draw the complete circuit diagram for each logic gates and their truth tables.

3.2 Experiment

1. Write a unified Verilog code that implements above mentioned logic gates and instantiates each logic module to the main module.
2. Use **3 slide switches (SW0, SW1, SW2)** for selecting the logic gate type, allowing for **7 different two-input gates**: 000 → AND, 001 → OR, 010 → NOT, 011 → XOR, 100 → XNOR, 101 → NAND, 110 → NOR.
3. Use **SW3 and SW4** as binary inputs (A and B) for the selected gate.
4. **LEDs** will light up to indicate the input status when switches are toggled. You may use the clock input of the FPGA to scan the switches (SW0, SW1, SW2).
5. The output of the selected gate will be displayed on **LED0**, showing whether the output is **ON (1)** or **OFF (0)**.

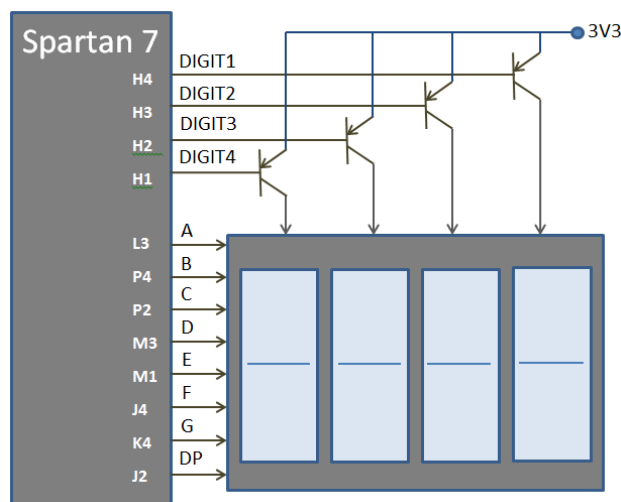


Fig. 1 Seven Segment Display Interfacing Circuit

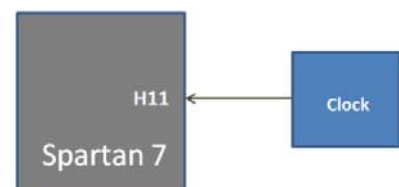


Fig. 2 Clock Interfacing Circuit